

Digital Design with Verilog

Finite State Machine

A. Sahu

Dept of Comp. Sc. & Engg.

Indian Institute of Technology Guwahati

Outline

- Finite State Machine
- Formal Definition
- State Machine Types
- FSM Implementation

Sequential Circuit: FSM

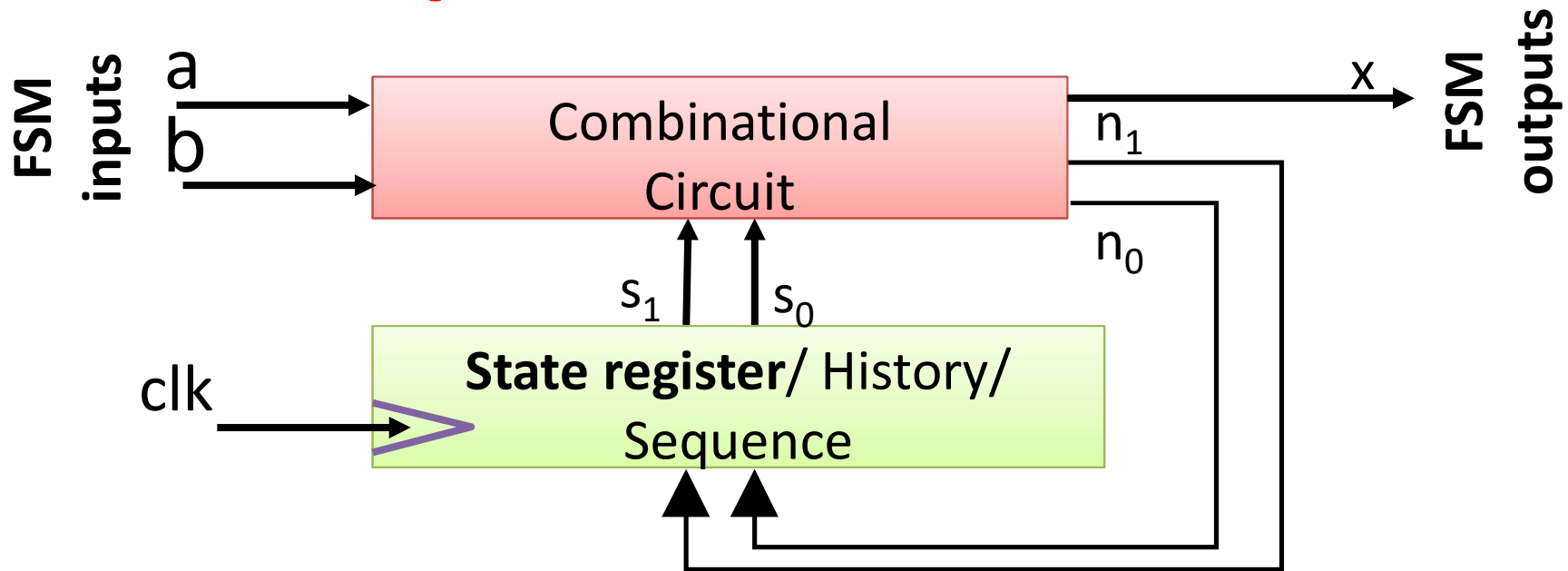
- Combinational Circuit : Formal Approach
 - Boolean Algebra
 - Circuit Minimization (K-Map, QMCluskey, Espresso...)
- Sequential Circuit : Formal Approach
 - Finite State Machine

Formally Describe/mathematically Describe

Boolean Algebra: Working of Combinational Circuit

Finite State Machine: Working of Sequential Circuit

Sequential Circuit: FSM



Formally Describe/mathematically Describe

Boolean Algebra: Working of Combinational Circuit
Finite State Machine: Working of Sequential Circuit

$X(t) = F(a(t), b(t), H)$
H is History/Sequence/State

Boolean Combinational Circuit

- Can you formally model All Combinational Circuit using Boolean Algebra?



Already Designed Sequential Circuit

- Flip Flops (RS, JK, T, D)
- Register (Shift, PIPO), Memory
- Counter : Sync, Modulo Counter
- Counter : Using Shift Register

Till now we have not used
formal Approach to design
these

Need a Better Way to Design Sequential Circuits

- Combinational circuit design process had two important things
 1. A formal way to describe desired circuit behavior
 - Boolean equation, or truth table
 2. A well-defined process to convert that behavior to a circuit
- **We need those things for sequence circuit design**

Sequential Circuit

- Can we model All Synchronous Sequential Circuits using some model?

**YES, with
Finite State Machine (FSM)**

Finite State Machine

- Finite-State Machine (FSM)
 - A way to describe desired behavior of sequential circuit
 - Akin to Boolean equations for combinational behavior
 - List states, and transitions among states

FSM Types

- Two main types of FSMs
 - Moore FSM : output is only function of state
 - Mealy FSM: output is function of state and inputs

Set Theoretic Description

Moore Machine is an ordered quintuple

$$M o o r e = (S, I, O, \delta, \lambda)$$

where

$$S = \{s_1, s_2, \dots, s_n\}$$

$$I = \{i_1, i_2, \dots, i_m\}$$

$$O = \{o_1, o_2, \dots, o_l\}$$

$$\delta = S \times I \rightarrow S$$

$$\lambda = S \rightarrow O$$

S= finite set of state ($\neq \phi$)

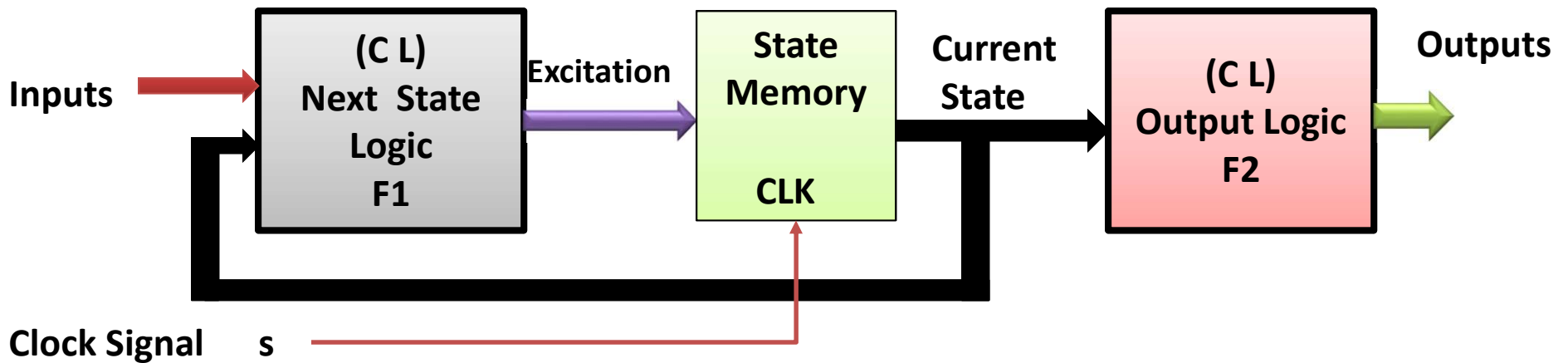
I = finite set of inputs ($\neq \phi$)

O=finite set of output ($\neq \phi$)

δ =next state function

λ = output function

Moore Machine: Moore FSM



Next state = $F_1(\text{current state, inputs})$
Output = $F_2(\text{current state})$

Set Theoretic Description

Mealy Machine is an ordered quintuple

$$\text{Mealy} = (S, I, O, \delta, \beta)$$

where

$$S = \{s_1, s_2, \dots, s_n\}$$

$$I = \{i_1, i_2, \dots, i_m\}$$

$$O = \{o_1, o_2, \dots, o_l\}$$

$$\delta = S \times I \rightarrow S$$

$$\beta = S \times I \rightarrow O$$

S= finite set of state ($\neq \phi$)

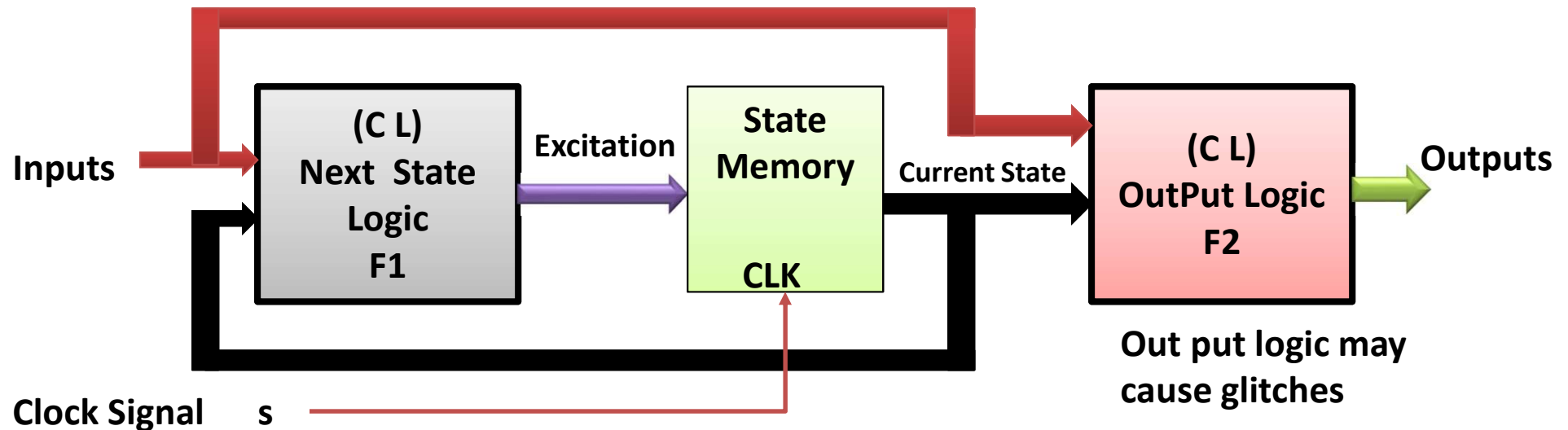
I = finite set of inputs ($\neq \phi$)

O=finite set of output ($\neq \phi$)

δ =next state function

λ = output function

Mealy Machine: Mealy FSM



State Storage= Set of n FFs
 2^n State can be stored

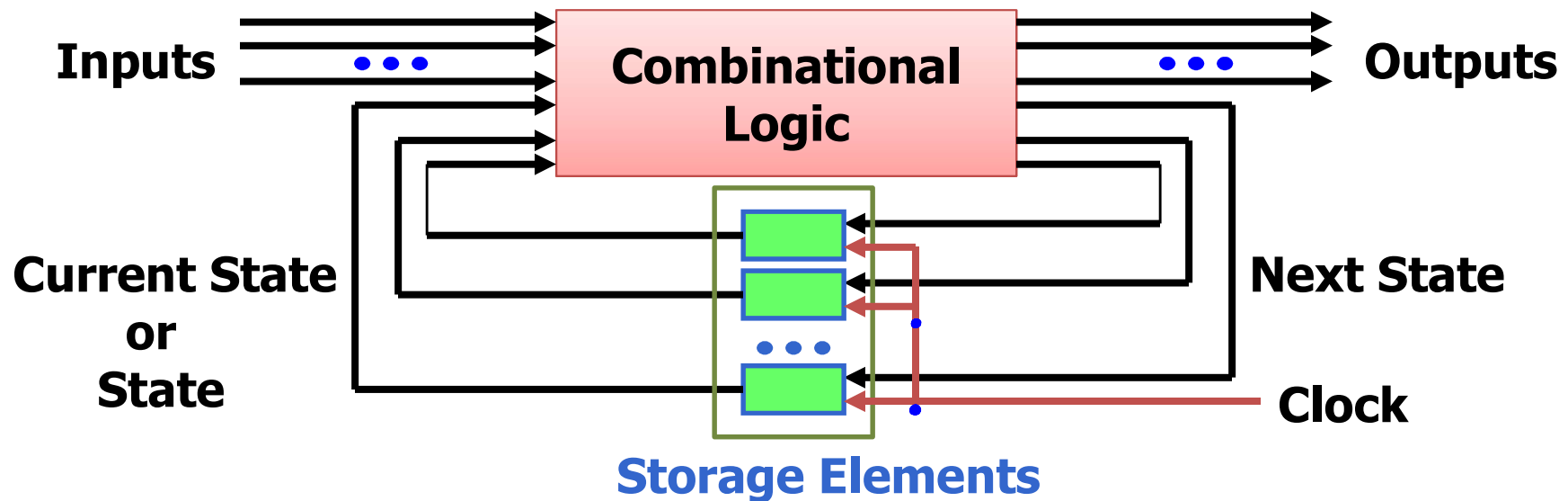
Next state = $F_1(\text{current state, inputs})$
Output = $F_2(\text{current state, inputs})$

Clocked synchronous FSM

- **Clocked**
 - All storage elements employ a clock input
 - (i.e. all storage elements are flip-flops)
- **Synchronous**
 - All of the flip flops use the same clock signal
- **FSM**
 - State machine is simply another name for sequential circuits.
 - Finite refers to the fact that the number of states the circuit can assume is finite

Clocked synchronous FSM structure

- **States:** Determined by possible values in sequential storage elements
- **Transitions:** Change of state
- **Clock:** Controls when state can change by controlling storage



Standard Procedure for Sequential Circuit Design

Sequential Circuit Design Process

- Step 1: Describe problem in HLL English like statements
- Step 2: Model the problem using FSM
 - **Step 2.A: (optional) Minimize FSM**
- Step 3: Represent the FSM in Tabular Form
- Step 4: Derive number of FF require

Sequential Circuit Design Process

- Step 5: State Encoding
 - **Step 6.A: Optional Encoding Optimization**
- Step 6: Derivation of
 - Next State : from Present State and Input
 - Output : from Present State (for Mealy FSM: including input)
- Step 7: Derive combination function for NS logic, OP logic
- Step 8: Realize FSM
 - by Connecting the NS, OP logic, State Reg and Clock

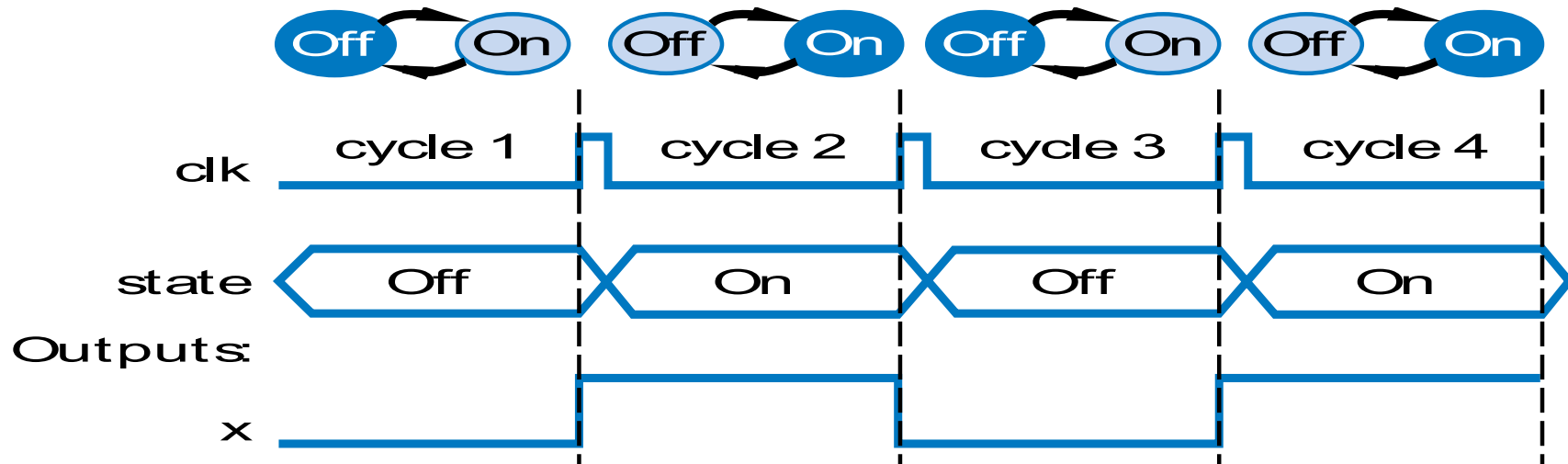
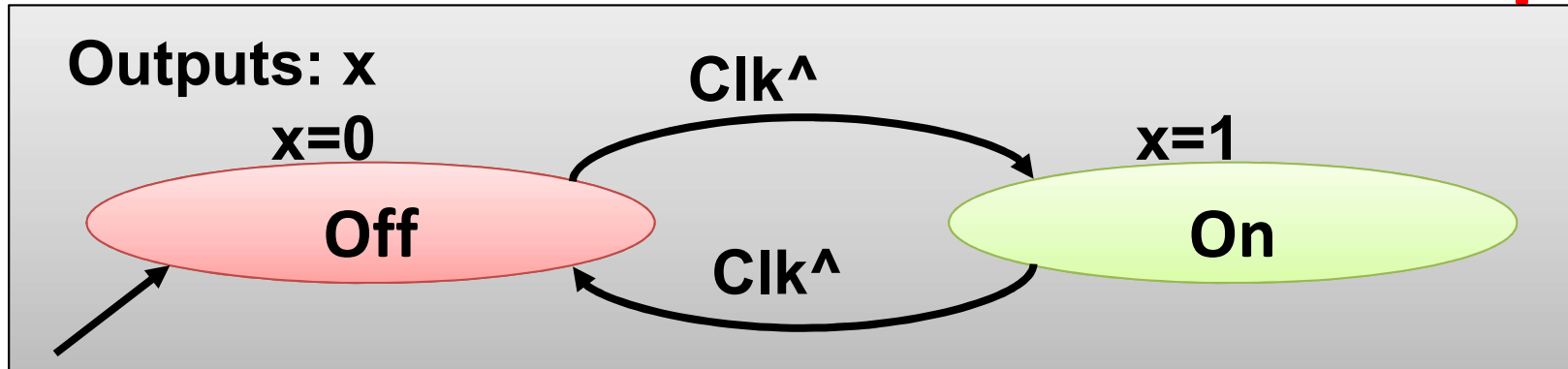
FSM Example 1

ON-OFF FSM

Finite State Machine: Example

- Step1: English like statement
 - Make x change toggle (0 to 1, or 1 to 0) every clock cycle
- Step2: Model the problem using FSM
 - Two states: “Off” ($x=0$), and “On” ($x=1$)
 - Transition on rising clock edge
 - from Off to On, or On to Off
 - Arrow with no starting state points to
 - initial state (when circuit first starts)

Moore Finite State Machine: Example



FSM

We often draw FSM graphically, known as *state diagram*

Can also use table (state table), or textual languages

Controller for On-Off

Step 3: Represent the FSM in Tabular Form

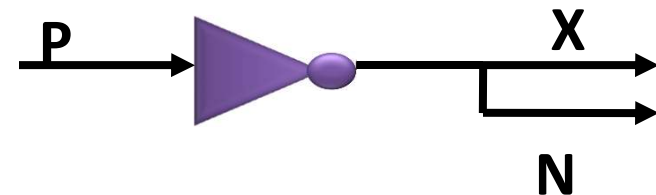
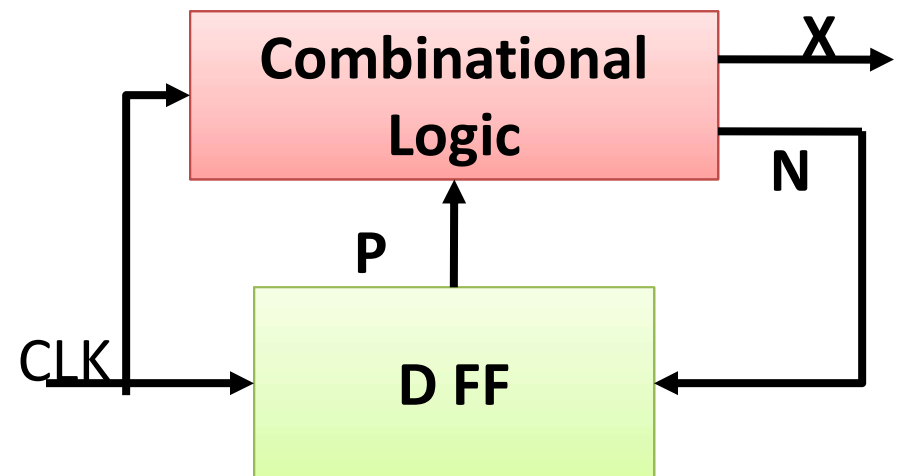
Input		Output	
CLK	PS	X	NS
RE 1	0	1	1
RE 1	1	0	0

Step 4: Derive number of FF require

Two States: 1 FF

Step 5: State Encoding

State 0: 0 State 1: 1



Rising Edge: Clock implicit

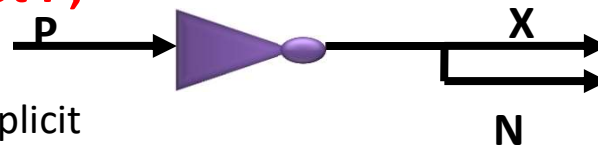
Controller for On-Off

Input		Output	
CLK	P	X	N
RE 1	0	1	1
RE 1	1	0	0

Step 6: Derivation of
 Next State : from Present State and Input
 Output : from Present State

Step 7: Derive combination function for
 NS logic, OP logic

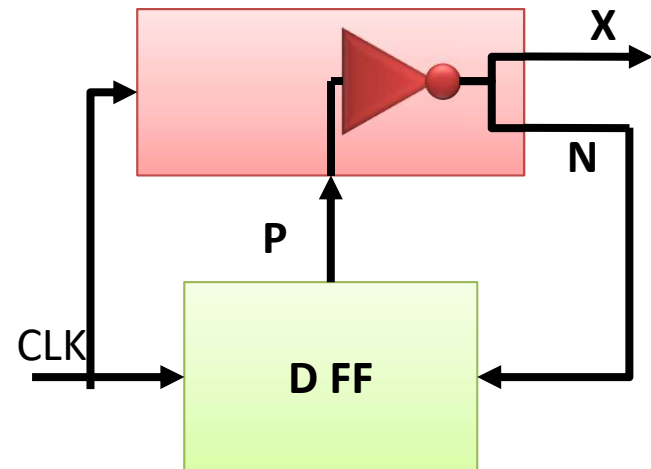
NS = not P; X=not P;



Rising Edge: Clock implicit

Step 8: Realize FSM

by Connecting the NS, OP logic, State Reg and Clock



Thanks